Algorithm – Part 1: introduction

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Motivation

- Some common problems in computer science:
  - *Given a sequence/list/set of integers, find the largest one*
  - *Given a set, list all of its subsets*
  - *Given a set of integers, put them in increasing order*
  - *Given a network, find the shortest path between two vertices*
- In computer science, the software area will lead you to mathematics
  - Hardware area will lead you to materials/physics

Computer Problems$_1$

- To solve the problems:
  - Construct a **model** that translates the problem into a **mathematical context**
    - Logic
    - Data structures: set, sequence, list
      - Operations, functions
      - Relations, graphs, trees
    - Status
      - Automata
      - Finite state machines
    - …

Computer Problems$_2$

- To solve the problems:
  - Build a method that will solve the general problem using the model
    - Ideally, what we need is a procedure that follows a sequence of steps that leads to the desired answer
    - This sequence of steps is called an **algorithm**

Definition: An **algorithm** is a finite sequence of precise instructions for performing a computation or for solving a problem

Properties of an Algorithm

- ***Input***: An algorithm has input values from a specified set.
- ***Output***: From the input values, the algorithm produces the output values from a specified set. The output values are the solution.
- ***Correctness***: An algorithm should produce the correct output values for each set of input values.
- ***Finiteness***: An algorithm should produce the output after a finite number of steps for any input.
- ***Effectiveness***: It must be possible to perform each step of the algorithm correctly and in a finite amount of time.
- ***Generality***: The algorithm should work for all problems of the desired form.

An Example

Task: describe an algorithm for finding the maximum element/term in a finite sequence of integers.

Solution: we perform the following steps:

1. Set the temporary maximum equal to the first integer in the sequence
2. Compare the next integer in the sequence to the temporary maximum, and if it is larger than the temporary maximum, set the temporary maximum equal to this integer
3. Repeat the previous step if there are more integers in the sequence
4. Stop when there are no integers left in the sequence. The temporary maximum at this point is the largest integer in the sequence

Specifying Algorithms

- Algorithms can be specified in different ways
  - English
  - *Pseudocode*
    - *Intermediate step between an English language description and a coding of these steps in a programming language*
    - *Can be used to implement an algorithm independently from the actual programming language*

Basic Statements in Pseudocode$_1$

- Procedure
  - Includes: name of the algorithm, Input variables
  - e.g. "**procedure** maximum (L: list of integers)"
- Assignments
  - e.g. max := a or max ← a
- If-else statement

if condition 1 **then** statement 1

else if condition 2 **then** statement 2

…

else statement n

Basic Statements in Pseudocode$_2$

- Loops
  - for loop:

```
for variable := initial value to final value
        block of statements
```

  - while loop

```
while condition
        block of statements
```

- Return

```
return x
```

Pseudocode for Previous Example

- The algorithm in pseudocode:

```
procedure max(a_1, a_2, …., a_n: integers)
    max := a_1
    for i := 2 to n
        if max < a_i then max := a_i
    return max
{max is the largest element}
```

Comments in braces

More Algorithms

- Two classes of problems will be studied

  1. *Searching Problems*: finding the position of a particular element in a list.

  2. *Sorting problems*: putting the elements of a list into increasing order.

Searching Algorithms

- Problem: "Locate an element x in a list of distinct elements $a_1, a_2, \ldots, a_n$, or determine that it is not in the list."
  - The solution to a searching problem is the location of the term in the list that equals *x*
  - We will study two searching algorithms
    - Linear Search
    - Binary Search

Linear Search Algorithm

- Examine all the elements in the sequence, one at a time, from the beginning to the end
- The algorithm in pseudocode:

```
procedure linear search(x:integer,
                        a_1, a_2, …,a_n: distinct integers)
i := 1
while (i ≤ n and x ≠ a_i)
        i := i + 1
if i ≤ n then location := i
else location := 0
return location
{location is the subscript of the term that equals
x, or is 0 if x is not found}
```

Binary Search$_1$

- ***Constraint***: can be used when the list has **terms occurring in a specific order** (words can be listed in lexicographic order)
- ***Methodology***:
  - Examine the middle term of the list
  - Since the list is ordered, we can then **shrink the search range to half** of the original sequence
    - *If the middle element is lower, the search proceeds with the upper half of the list.*
    - *If it is not lower, the search proceeds with the lower half of the list (through the middle position).*
  - Repeat this process until we have a list of size 1.

Binary Search$_2$

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8 (**why???**). The value in the 8th position is 10. Since 19 > 10, further search is restricted to positions 9 through 16.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

2. The midpoint of the list (positions 9 through 16) is now the 12th (**why???**) position with a value of 16. Since 19 > 16, further search is restricted to the 13th position and above.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

Binary Search$_3$

3. The midpoint of the current list is now the 14th position with a value of 19 (**if you know how to use "break" in loops, we can terminate here**). Since 19 $\not>$ 19, further search is restricted to the portion from the 13th through the 14th positions .

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

4. The midpoint of the current list is now the 13th position with a value of 18.
   Since 19> 18, search is restricted to the portion from the 14th position through the 14th.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

5. Now the list has a single element and the loop ends. Since 19=19, the location 14 is returned.

Binary Search$_4$

- The algorithm in pseudocode:

```
procedure binary search(x: integer, a_1,a_2,…, a_n: increasing integers)
        i := 1      {i is the left endpoint of interval}
        j := n      {j is right endpoint of interval}
        while i < j
                    m := ⌊(i + j)/2⌋
                    if x > a_m then i := m + 1
                    else j := m
        if x = a_i then location := i
        else location := 0
        return location
{location is the subscript i of the term a_i equal to x, or 0 if x is not found}
```

Binary Search$_5$

- Keys of Binary Search
  - How to calculate the index of the mid point
    - When there are even number of terms: choose the lower index or higher index?
  - Avoid infinite loops
    - In each iteration, we need to make sure the search range is shrinking
      - If the search range remains the same, we will be trapped in infinite loops, and the program won't stop
    - The right boundary or left boundary of the search range needs to be updated
      - How to update the boundary is also related to the choice of the mid point when there are even number of terms
    - This could be a typical mistake in Binary Search

Sorting

- ***Goal***: to *sort* the elements of a list is to put them in increasing order (numerical order, alphabetic, and so on).
  - The solution to a sorting problem is a sorted list presented in a particular order
  - We will study two sorting algorithms
    - Bubble Sort
    - Insertion Sort

Bubble Sort$_1$

- ***Bubble sort*** makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.
- The algorithm in pseudocode:

procedure *bubbleSort* ($a_1,\ldots,a_n$: real numbers with $n \geq 2$)

  for i := 1 to $n-1$

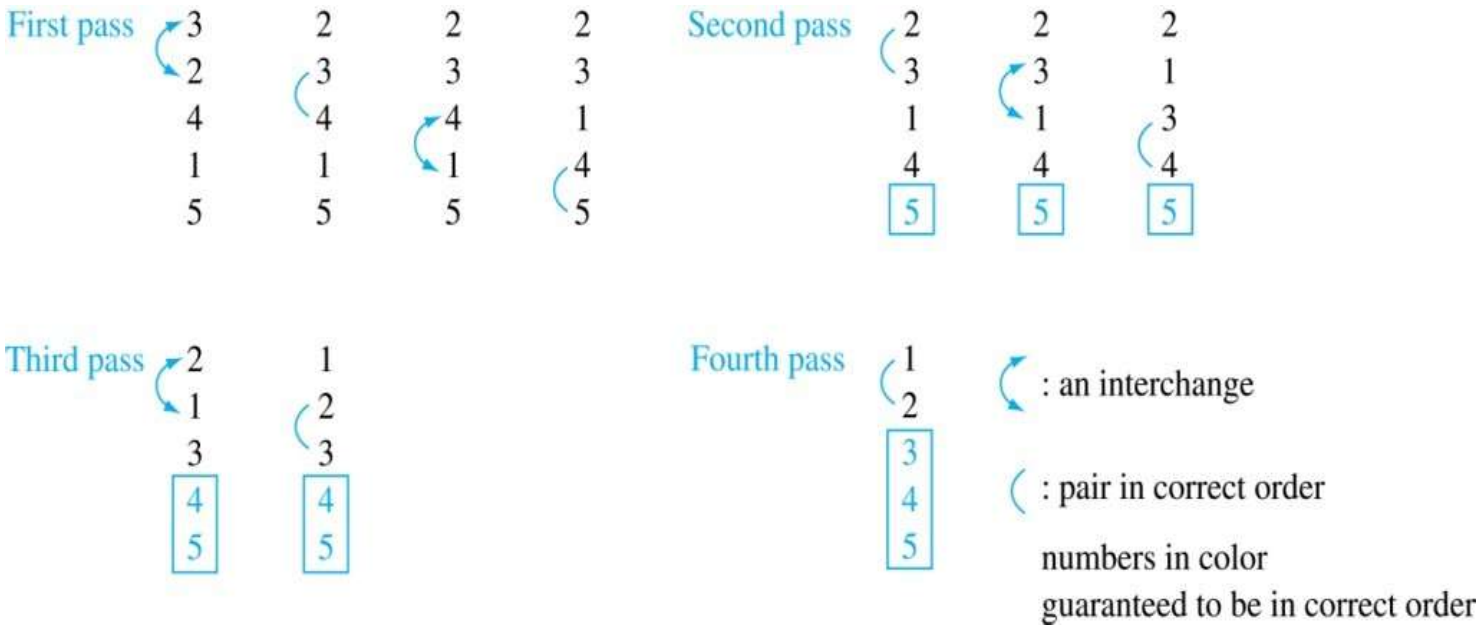
    for j := 1 to $n-i$

      if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}

{$a_1,\ldots, a_n$ is now in increasing order}

Bubble Sort$_2$

- **Example**: Show the steps of bubble sort with 3 2 4 1 5



In each subsequent pass, an additional element is put in the correct position (close to the bottom: largest ones).

Insertion Sort$_1$

- ***Insertion sort*** iterates through all the terms in the list. In n-th iteration, the n-th term would be placed to its right place in the first n terms, so that after this iteration the first n terms are sorted.
- First term can be skipped
- Linear search is used to find the correct position.

- The algorithm in pseudocode:

```
procedure insertionSort (a_1,…,a_n:
        real numbers with n ≥ 2)
        for j := 2 to n
            i := 1
            while a_j > a_i
                i := i + 1
            m := a_j
            for k := 0 to j – i – 1
                a_{j-k} := a_{j-k-1}
            a_i := m
{Now a_1,…,a_n is in increasing order}
```

Insertion Sort$_2$

- **Example**: Show all the steps of insertion sort with the input: 3 2 4 1 5

  2 3 4 1 5

  (*first two positions are interchanged*)

  2 3 4 1 5

  (*third element remains in its position*)

  1 2 3 4 5

  (*fourth is placed at beginning*)

  1 2 3 4 5

  (*fifth element remains in its position*)

Practice 1

- 1. List all steps used to search for 9 in the sequence: 1, 3, 4, 5, 6, 8, 9, 11
  - How about searching for 2?

Practice 2

- 1. List all steps used to search for 9 in the sequence: 1, 3, 4, 5, 6, 8, 9, 11
  - i=0, j=7, m=3: 1, 3, 4, 5, 6, 8, 9, 11
  - i=4, j=7, m=5: 1, 3, 4, 5, 6, 8, 9, 11
  - i=6, j=7, m=6: 1, 3, 4, 5, 6, 8, 9, 11
  - return 6 (the index of 9)
- Search for 2
  - i=0, j=7, m=3: 1, 3, 4, 5, 6, 8, 9, 11
  - i=0, j=3, m=1: 1, 3, 4, 5, 6, 8, 9, 11
  - i=0, j=1, m=0: 1, 3, 4, 5, 6, 8, 9, 11
  - return -2 (2 can be inserted at index 1, so return -(1+1)

Practice 3

- 2. Apply insertion sort to: 6, 2, 3, 1, 5, 4

Practice 4

- 2. Apply insertion sort to: 6, 2, 3, 1, 5, 4
  - 6, 2, 3, 1, 5, 4
    - m=2 is saved: 6, _, 3, 1, 5, 4;
    - Push numbers back: _, 6, 3, 1, 5, 4;
    - Insert m to the insertion point: 2, 6, 3, 1, 5, 4
  - 2, 6, 3, 1, 5, 4
    - 2, 6, _, 1, 5, 4; 2, _, 6, 1, 5, 4; 2, 3, 6, 1, 5, 4
  - 2, 3, 6, 1, 5, 4
    - 2, 3, 6, _, 5, 4; _, 2, 3, 6, 5, 4; 1, 2, 3, 6, 1, 5, 4
  - 1, 2, 3, 6, 5, 4
    - 1, 2, 3, 6, _, 4; 1, 2, 3, _, 6, 4; 1, 2, 3, 5, 6, 4
  - 1, 2, 3, 5, 6, 4
    - 1, 2, 3, 5, 6, _; 1, 2, 3, _, 5, 6; 1, 2, 3, 4, 5, 6
  - 1, 2, 3, 4, 5, 6

Practice 5

- 3. Describe an algorithm that locates the last occurrence of the largest element in a finite list of integers, where the integers in the list are not necessarily distinct.

Practice 6

- 3. Describe an algorithm that locates the last occurrence of the largest element in a finite list of integers, where the integers in the list are not necessarily distinct.

```
Procedure last_largest (a1, …, an: integers)
  max := a1
  location := 1
  for i := 2 to n
    if max <= ai then
      max := ai
      location := i
  return location
```

Extension 1

- 1. Find the first occurrence of the target in a sorted array of integers, where duplicates may exist.
  - How about the last occurrence of the target?

Extension 2

- 2. Describe an algorithm that determines whether a function from a finite set of integers to another finite set of integers is onto.